

LETTER

Formal and Empirical Analysis of Authorization Decision Inconsistency in SAGA

TODO AUTHOR[†], ??

SUMMARY We analyze an order-dependent authorization inconsistency in SAGA: a later wildcard allow can override a more-specific deny because the matcher fails to retain the best pattern. Controlled production consumers propagate this decision to token issuance/storage. Under the fixed divergence, ProVerif reports symbolic ChatAccept reachable with prior TokenIssue.

key words: *authorization, access control, formal verification, AI agents, ProVerif*

1. Introduction

SAGA [1] uses contact policies and access-control tokens to authorize communication between registered agents. Authentication establishes identity but does not imply permission under the receiver's contact policy.

We identify an order-dependent inconsistency in the production matcher. Under documented most-specific semantics, a specific deny should override a wildcard allow. Instead, the matcher returns the later wildcard allowance; reversing the rule order returns denial.

We examine two consequences. Controlled production consumers propagate the inconsistent decision through the Provider and receiver to token issuance/storage. Separately, a symbolic model instantiated with the same fixed divergence reaches ChatAccept; ProVerif also reports its correspondence to prior TokenIssue as true. The empirical execution stops at token storage, before message acceptance.

Our contributions are (1) identifying and explaining the matcher inconsistency, and (2) characterizing its consequences through controlled production-consumer propagation and conditional symbolic analysis.

(1) Related work.

Authorization-policy work studies policy semantics and analysis [2], [3]; ProVerif supports symbolic protocol analysis [4]. We relate a concrete matcher disagreement to both forms of analysis.

2. Authorization Decision Inconsistency

(1) Workflow and policy semantics.

An initiator requests contact through SAGA's Provider. The receiver's ordered policy maps identity patterns to integer budgets. Provider and receiver independently call the production matcher on the initiator. The Provider gates one-time

Table 1 Archived matcher observations under two rule orders.

Rule order	Expected budget	Observed budget
Specific deny, then wildcard allow	-1	10
Wildcard allow, then specific deny	-1	-1

key (OTK) release; the receiver rechecks its policy before token issuance.

For a fixed policy and valid identity, SAGA v2, Section IV-D.1 [1], specifies the most-specific matching rule; the implementation scores patterns with `aid_specificity()`. A positive budget permits the request; a negative budget denies it. Equal-score conflicts, changing policies, and zero budgets are outside the cases.

(2) Root cause and observation.

The tested production matcher is `match()` in `saga/common/contact_policy.py`, revision 7372111 of the study repository.[†] Its `best_pattern` starts at `None`, whose specificity is -1. A matching rule with greater specificity updates the budget but never `best_pattern`. Thus later matching rules still compare against -1, allowing even a less-specific wildcard to overwrite the budget. For the recorded identity `alice@example.com:agent`, `aid_specificity()` scores pattern `alice@example.com:*` as 70 and `*` as 0; their budgets are -1 and 10, respectively.

An independent finite specification evaluator selects the specific denial for both orders in Table 1. When specific deny precedes wildcard allow, the specification denies with budget -1, but the production matcher returns 10 and permits. The reverse order returns the specified denial, establishing order dependence. These calls establish only a local decision inconsistency, not deployment exploitation, message acceptance, or whole-system failure.

3. Formal Consequence Analysis

(1) Decision input.

The symbolic model is instantiated with the recorded divergence: the specification denies the request, while the observed implementation permits it. ProVerif consumes these fixed facts; it does not execute Python or the production

[†]TODO: affiliation and postal address.

DOI: 10.1587/transfun.E0.A.1

[†]github.com/g1020268262-a11y/saga

matcher, discover the inconsistency, or prove implementation refinement.

(2) Model assumptions.

The initiator has a valid registered identity, follows the prescribed symbolic peer process, and cannot alter the receiver's fixed policy; reachability is not an arbitrary-adversary reachability claim. The Provider, CA, effective cryptographic checks, and honest private keys are trusted. A private symbolic channel abstracts authenticated, confidential transport.

In the symbolic receiver, `TokenIssue( $R, I, t$ )` occurs after inserting receiver R , initiator I , encrypted token t , and I 's public access-control key into `ActiveToken`, before sending t . For a message m received on the private channel, `ChatAccept( $R, I, t, m$ )` occurs only after checking the fixed initiator identity and an `ActiveToken` entry matching receiver, initiator, presented token, and sender public key.

(3) Queries and results.

The run reports `ChatAccept` reachable under the fixed divergence. Separately, ProVerif reports the following non-injective correspondence as true:

$$\text{ChatAccept}(R, I, t, m) \implies \text{TokenIssue}(R, I, t)$$

It requires prior `TokenIssue` for the same receiver R , initiator I , and encrypted token t used by `ChatAccept`; it does not establish intended policy permission. Issuance appears in the acceptance witness; no standalone `TokenIssue` reachability query exists.

The separate empirical and symbolic executions are compared only at the decision/token lifecycle level; no object-level refinement is established. Only the symbolic execution reaches message acceptance.

4. Consumer-Side Propagation Validation

(1) Environment.

The harness executes production matcher calls in Provider /access and the receiver handler, with OTK release/consumption, token generation, and `active_tokens` storage. Fixtures replace registration, certificate services, and TLS transport. Execution stops only at `receive_conversation`, after token sending and storage.

(2) Cases and controls.

Provider and receiver use identical policies. The divergent case places specific deny (−1) before wildcard allow (10): the specification denies, while the production implementation permits. The controls use specific then wildcard allowances (10 each), or wildcard allow (10) followed by specific deny (−1). Each case ran once with fresh state and one OTK. The allow/allow (A/A) positive control reaches token sending/storage; the deny/deny (D/D) Provider-blocking control returns HTTP 403, releases no OTK, leaves state unchanged, and never reaches the receiver, so it does not test receiver rejection.

Table 2 Divergent-case production propagation; message acceptance was not executed.

Evidence point	Divergent case result
Specification	Deny (−1)
Provider matcher	Allow (10)
Provider /access	HTTP 200; OTK released
Receiver matcher	Allow (10)
Receiver	OTK consumed
Endpoint	Token issued and stored

(3) Result and evidence.

Table 2 traces the divergent decision through production consumers. Both matchers return 10. The Provider returns HTTP 200 and releases its OTK; the receiver consumes the OTK, generates and sends one token, and stores it in `active_tokens`.

The empirical endpoint is token issuance/storage, not message acceptance: no application message is sent. No `receive_conversation` logic runs. Neither `local_agent.run`, LLM calls, nor tool execution occurs.

5. Scope and Conclusion

Evidence covers one recorded SAGA case and tested policies, not arbitrary-policy correctness or Python–ProVerif refinement. Empirical execution stops before message acceptance; no deployed exploit is established. The matcher inconsistency propagates to token issuance/storage in controlled production consumers; separately, symbolic `ChatAccept` is reachable under the fixed divergence.

References

- [1] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "SAGA: A security architecture for governing AI agentic systems." arXiv preprint arXiv:2504.21034v2, 2025. Version 2, revised 29 August 2025.
- [2] M.Y. Becker, C. Fournet, and A.D. Gordon, "Design and semantics of a decentralized authorization language," 20th IEEE Computer Security Foundations Symposium (CSF), pp.3–15, IEEE, 2007.
- [3] K. Fislser, S. Krishnamurthi, L.A. Meyerovich, and M.C. Tschantz, "Verification and change-impact analysis of access-control policies," 27th International Conference on Software Engineering (ICSE), 2005.
- [4] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," Foundations and Trends in Privacy and Security, vol.1, no.1–2, pp.1–135, 2016.